Chapter 78

Understanding Image Channels

This chapter seeks to demystify how Fusion handles image channels and, in the process, show you how different nodes need to be connected to get the results you expect.

It also explains the mysteries of premultiplication, and presents a full explanation of how Fusion is capable of using and even generating auxiliary data.

Contents

Channels in Fusion

Fusion introduces some innovative ways of working with the many different channels of image data that modern compositing workflows encompass. This chapter's introduction to color and data channels and how they're affected by different nodes and operations is a valuable way to begin the process of learning to do paint, compositing, and effects in Fusion.

If you're new to compositing, or you're new to the Fusion workflow, you ignore this chapter at your peril, as it provides a solid foundation to understanding how to predictably control image data as you work in this powerful environment.

Types of Channels Supported by Fusion

Digital images can be divided into separate streams of image data called Channels, each of which carries a specific kind of image data. Nodes that perform different image processing operations typically expect specific channels to provide predictable results. You are probably familiar with the three standard color channels of red, green, and blue, but there are many others. This section describes the different kinds of channels that Fusion supports.

RGB Color channels

The red, green, and blue channels of any still image or movie clip combine additively to represent everything we can see via visible light. Each of these three channels is a grayscale image when seen by itself. When combined additively, these channels represent a full-color image.

Alpha Channels

An alpha channel is an embedded fourth channel that defines different levels of transparency in an RGB image. Alpha channels are typically embedded in RGB images that are generated from computer graphics applications. In Fusion, white denotes solid areas, while black denotes transparent areas. Grayscale values range from more opaque (lighter) to more transparent (darker).

If you're working with an imported alpha channel from another application for which these conventions are reversed, never fear. Every node capable of using an alpha channel is also capable of inverting it.

Single-Channel Masks

While similar to alpha channels, mask channels are single channel images, external to any RGB image and typically created by Fusion within one of the available Mask nodes. Mask nodes are unique in that they propagate single-channel image data that defines which areas of an image should be solid and which should be transparent. However, masks can also define which parts of an image should be affected by a particular operation, and which should not. Mask channels are designed to be connected to specific mask inputs of nodes including Effect Mask, Garbage Mask, and Solid Mask inputs.

Auxiliary Channels

Auxiliary channels (covered in more detail later in this chapter), describe a family of special-purpose image data that typically expose 3D data in a way that can be used in 2D composites. For example, Z-Depth channels describe the depth of each pixel in an image along a Z axis, while an XYZ Normals channel describes the orientation (facing up, facing down, or facing to the left or right) of each pixel in an image. Auxiliary channel data is generated by rendering 3D images, so it usually accompanies or is embedded with RGB images generated by 3D modeling and animation applications.
These channels can also be generated from within Fusion via the Renderer 3D node, which outputs a 3D scene that you've assembled and lit as 2D RGBA channels, with optionally accompanying auxiliary channels.

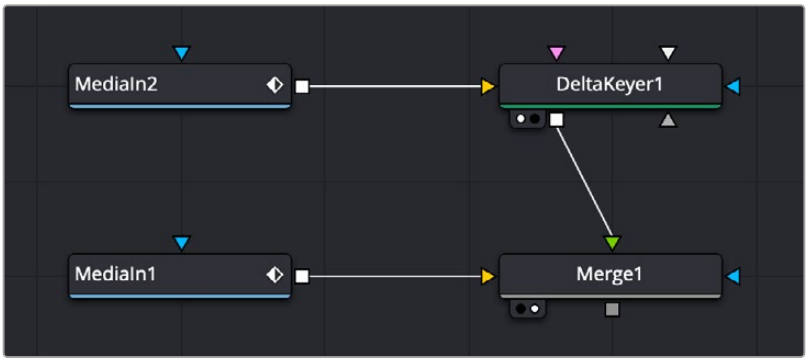
The reason to use auxiliary data is that 3D rendering is computationally expensive and time-consuming, so outputting descriptive information about a 3D image that's been rendered empowers compositing artists to make sophisticated alterations in 2D. You can add motion blur, perform relighting, and composite with depth information faster than re-rendering the 3D source material over and over.

TIP: You can view any of a node's channels in isolation using the Color drop-down menu in the viewer. Clicking the Color drop-down menu reveals a list of all channels within the currently selected node, including red, green, blue, or auxiliary channels.

Fusion Node Connections Carry Multiple Channels

The connections that pass image data from one node to the next in Fusion's Node Editor are capable of carrying multiple channels of image data. That means that a single connection may route RGB, or RGBA, or RGBAZ-Depth, or even just Z-Depth, depending on the Input you connect to and the node's function.

In the following example, the two MediaIn nodes output RGB data. However, the Delta Keyer creates an alpha channel and combined it with MediaIn2's RGB image. The RGB-A of the Delta Keyer becomes the foreground image that the Merge node can use to create a two-layer composite.



The alpha channel generated by the Delta key is used for compositing by the foreground input of the Merge node.

NOTE: Node trees shown in this chapter may display MediaIn nodes found in DaVinci Resolve's Fusion page; however, Fusion Studio Loader nodes are interchangeable unless otherwise noted.

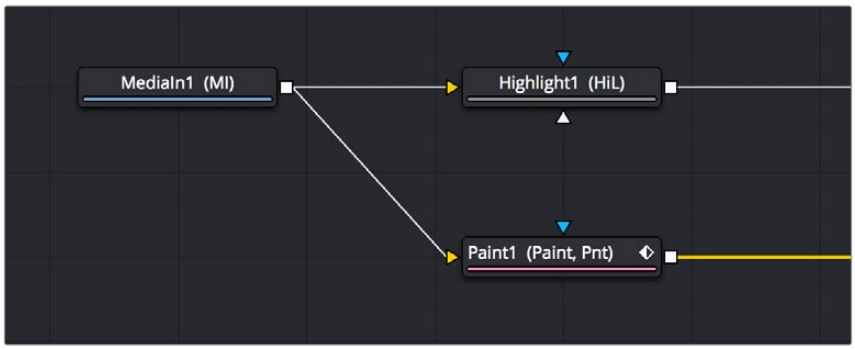
Running multiple channels through single connection lines makes Fusion node trees simple to read, but it also means you need to keep track of which nodes process which channels to make sure that you're directing the intended image data to the correct operations.

Node Inputs and Outputs

By default, MediaIn nodes in the Fusion page and Loader nodes in Fusion Studio output RGBA channels. When you connect one node's output to another node's input, the active channels are passed from the upstream node to the downstream node, which then processes the image according to that node's function. Only one node output can be connected to a node input at a time. In this simple example, a MediaIn node's output is connected to the input of a Highlight node to create a sparkly highlight effect.

MediaIn node connected to a Highlight node connected to MediaOut node in the Fusion page.

When connecting nodes, a node's output carries the same channels no matter how many times the output is "branched." You cannot send one channel out on one branch and a different channel out on another branch.

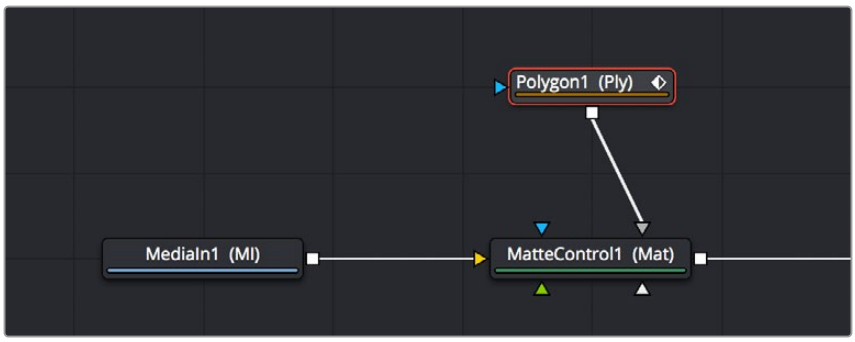


The MediaIn node's output is branched but carries the same RGB channels to both inputs.

Using Multiple Inputs

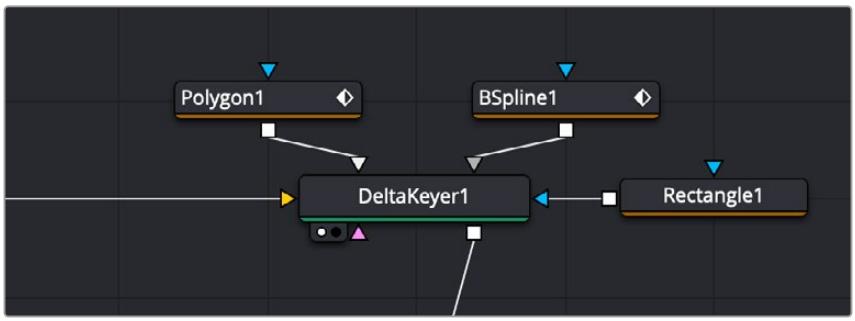
Most nodes have two inputs, one for RGBA and another for an effect mask that can be optionally used to limit the effect of that node to a particular part of the image. However, some nodes have three or even more inputs, and it's important to make sure you connect the correct image data to the appropriate input to obtain the desired result. If you connect one node's output to another node's input and nothing happens, chances are you've connected to the wrong input.

For example, the MatteControl node has a background input and a foreground input, both of which accept RGBA channels. However, it also has SolidMatte, GarbageMatte, and EffectsMask inputs that accept alpha or mask channels to modify the transparency of the Node's output. If you want to perform the extremely common operation of using a MatteControl node to create an alpha channel using a Polygon node for rotoscoping an image, you need to make sure that you connect the Polygon node to the GarbageMatte input to obtain the correct result. The GarbageMatte input is automatically set to alter the alpha channel of the foreground image. If you connect to any other input, your Polygon mask may not produce expected results.



Polygon node connected to the GarbageMatte input of a MatteControl node for rotoscoping

In another example, the DeltaKeyer node has a primary input (labeled “Input”) that accepts RGBA channels, but it also has three Matte inputs. These SolidMatte, GarbageMatte, and EffectsMask inputs on the Delta Keyer accept alpha or mask channels to modify the matte being extracted from the image in different ways.



The DeltaKeyer combines the multiple mask nodes in different ways.

If you position your pointer over any node’s input or output, a tooltip appears in the Tooltip bar at the bottom of the Fusion window, letting you know what that input or output is for, to help guide you to using the right input for the job. If you pause for a moment longer, another tooltip appears in the Node Editor itself.

(Left) The node input’s tooltip in the Tooltip bar,
(Right) The node tooltip in the Node Editor

Connecting to the Correct Input

When you’re connecting nodes, pulling a connection line from the output of one node and dropping it right on top of the body of another node makes a connection to the default input for that node, which is typically the “Input” or “Background” input.

Side by side, dropping a connection on a node’s body to connect to that node’s primary input

However, if you drop a connection line right on top of a specific input, then you'll connect to that input, so it's important to be mindful of where you drop connection lines as you wire up different node trees together.

Side by side, dropping a connection on a specific node input, note how the inputs rearrange themselves afterwards to keep the node tree tidy-looking

TIP: If you hold the Option key down while you drag a connection line from one node onto another, and you keep the Option key held down while you release the pointer's button to drop the connection, a menu appears that lets you choose which specific input you want to connect to, by name.

Inputs Require Specific Channels

Usually, you're prevented from connecting a node's output to another node or node input that's not compatible with it. For example, if you try to connect a Text3D node's output directly to the input of a regular Merge node, it won't work; 3D nodes do not produce RGB images, they produce 3D geometry data, so you must first connect to a Renderer3D node that creates the RGB output appropriate for 2D compositing operations.

In other cases, connecting the wrong image data to the wrong node input won't give you any error, it simply fails to produce the result you were expecting, necessitating you to troubleshoot the composition. If this happens to you, check the Fusion Effects section of this manual to see if the node you're trying to connect to has any limitations as to how it must be attached.

TIP: This chapter tries to cover many of the easy-to-miss exceptions to node connection that are important for you to know, so don't skim too fast.

Always Connect the Background Input First

Many nodes combine images in different ways, using "background" and "foreground" inputs. This includes the Merge node, the Matte Control node, and the Channel Booleans node, to cite common examples. The color of inputs on a node can help you make the right corrections. For instance, background inputs are always orange, and foreground inputs are always green.

When you first connect any node's output to a multi-input node, you usually want to connect the background input first. This is handled for you automatically when you first drop a connection line onto the body of a new multi-input node. The orange-colored background input is almost always connected first (the exception is Mask nodes, which always connect to the first available Mask input). This is good because you want to get into the habit of always connecting the background input first.

If you connect to only one input of a multi-input node and you don't connect to the background input, you may find that you don't get the results you wanted. This is because each multi-input node expects that the background is connected before anything else so that the internal connections and math used by that node can be predictable.

TIP: The only node to which you can safely connect the foreground input prior to the background input is the Dissolve node, which is a special node that can be used to either dissolve between two inputs, or automatically switch between two inputs of unequal duration.

Node Colors Tell You Which Nodes Go Together

Each node in Fusion accomplishes a single type of effect or operation. These single-purpose nodes make it easier to decipher a complex composition when examining its node tree. Single-purpose nodes also make it easier to focus on fine-tuning specific adjustments, one at a time, when assembling an ever-growing tree.

Because each Fusion node has a specific function, they're categorized by type to make it easier to keep track of which nodes require what types of image channels as input, and what image data you can expect each node to output. These general types are described here.

A node tree showing the main categories of node colors

Blue MediaIn and Loader Nodes, and Green Generator Nodes

Blue MediaIn nodes and blue Loader nodes add clips to a composite, and green Generator nodes create images. Both types of nodes output RGBA channels (depending on the source and generator), and may optionally output auxiliary channels for doing advanced compositing operations.

Because these are sources of images, both kinds of nodes can be attached to a wide variety of other nodes for effects creation besides just 2D nodes. For example, you can also connect MediaIn nodes to Image Plane 3D nodes for 3D compositing, or to pEmitter nodes set to “Bitmap” for creating different particle systems. Green Generator nodes can be similarly attached to many different kinds of nodes, for example, attaching a FastNoise node to a Displace 3D node to impose undulating effects to 3D shapes.

Shape nodes are also green, although they must be attached to a specialized set of gray modifier and render nodes (all of which begin with the letter “s” and appear in the Shape category of the Effects Library).

2D Processing Nodes, Color Coded by Type

These encompass most 2D processing and compositing operations in Fusion, all of which process RGBA channels and pass along auxiliary channels. These include:

- Orange Blur nodes
- Olive Color Adjustment nodes (color adjustment nodes additionally concatenate with one another)
- Pink Paint nodes
- Dark orange Tracking nodes
- Tan Transform node (transform nodes additionally concatenate with one another)
- Teal VR nodes
- Dark brown Warp nodes
- Gray, which includes Compositing nodes as well as many other types.

Additionally, some 2D nodes such as Fog and Depth Blur (in the Deep Pixel category) accept and use auxiliary channels such as Z-Depth to create different perspective effects in 2D.

> **TIP:** Two 2D nodes that specifically don’t process alpha channel data are the Color Corrector node and the Gamut node. The Color Correction node lets you color correct a foreground layer to match a background layer without affecting an alpha channel. The Gamut node lets you perform color space conversions to RGB data from one gamut to another without affecting the alpha channel.

Purple Particle System Nodes

These are nodes that connect to create different particle systems, and they’re incompatible with other kinds of nodes until you add a pRender node that outputs 2D RGBA and auxiliary data that can be composited with other 2D nodes and operations.

Dark Blue 3D Nodes

These are 3D operations, which generate and manipulate 3D data (including auxiliary channels) that are incompatible with other kinds of nodes until processed via a Renderer 3D node, which then outputs RGBA and auxiliary data.

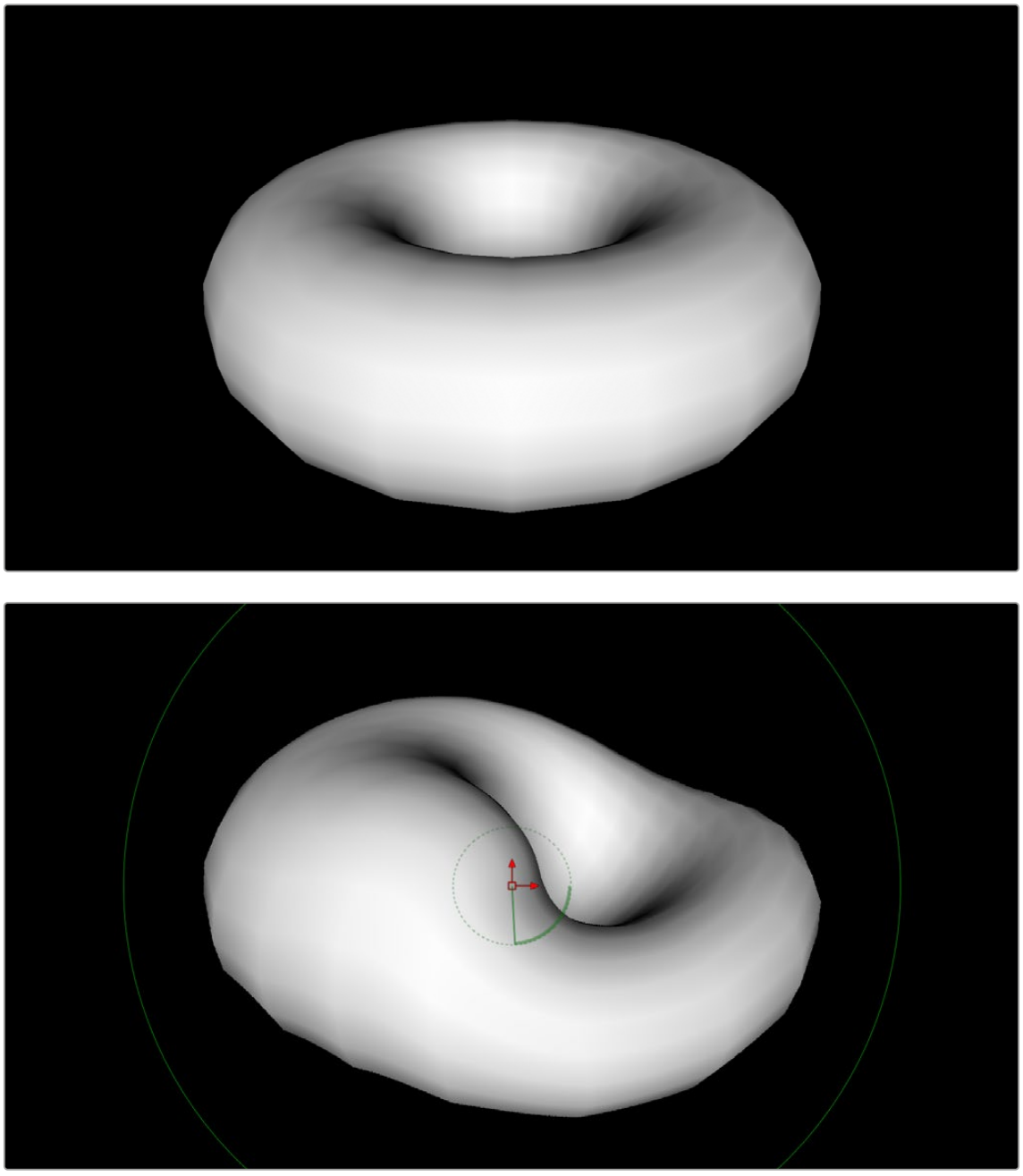
Brown Mask Nodes

Masks output single-channel images that can only be connected to one another (to combine masks) or to specified Mask inputs. Masks are useful for defining transparency (Alpha masks), defining which parts of an image should be cropped out (Garbage masks), or defining which parts of an image should be affected by a particular node operation (Effects masks).

Using Channels in a Composition

When you connect one node's Output to another node's Input, you feed all of the channels that are output from the upstream node to the downstream node. 2D nodes, which constitute most simple image processing operations in Fusion, propagate all channel data from node to node, including RGB, alpha, and auxiliary channels, regardless of whether or not that node actually uses or affects a particular channel.

2D nodes also typically operate upon all channel data routed through that node. For example, if you connect a node's output with RGBA and XYZ Normals channels to the input of a Vortex node, all channels are equally transformed by the Size, Center, and Angle parameters of this operation, including the alpha and XYZ normals channels, as seen in the following screenshot.



(Top) The Normal Z channel output by a rendered torus, (Bottom) The Normal Z channel after the output is connected to a Vortex node; note how this auxiliary channel warps along with the RGB and A channels

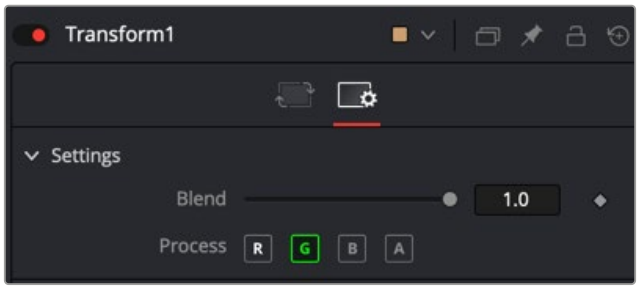
This is appropriate because in most cases, you want to make sure that all channels are transformed, warped, or adjusted together. You wouldn't want to shrink the image without also shrinking the alpha channel along with it, and the same is true for most other operations.

On the other hand, some nodes deliberately ignore specific channels when it makes sense. For example, the Color Corrector and Gamut nodes, both of which are designed to alter RGB data specifically, do not affect auxiliary channels. This makes them convenient for color-matching foreground and background layers you're compositing, without worrying that you're altering the depth information accompanying that layer.

TIP: If you're doing something exotic and you actually want to operate on a channel that's usually unaffected by a particular node, you can always use the Channel Booleans node to reassign the channel. When doing this to a single image, it's important to connect that image to the background input of the Channel Booleans node, so the alpha and auxiliary channels are properly handled.

Channel Limiting

Most nodes have a set of Red, Green, Blue, and Alpha buttons in the Settings tab of that node's Inspector. These buttons let you exclude any combination of these channels from being affected by that node.



The channel limiting buttons in the Settings panel of a Transform node, so only the Green channel is affected

For example, if you wanted to use the Transform node to affect only the green channel of an image, you can turn off the Red, Blue, and Alpha buttons. As a result, the green channel is processed by this operation, and the red, blue, and alpha channels are copied straight from the node's input to the node's output, skipping that node's processing to remain unaffected.

Transforming only the green color channel of the image with a Transform effect

Skipping Channel Processing

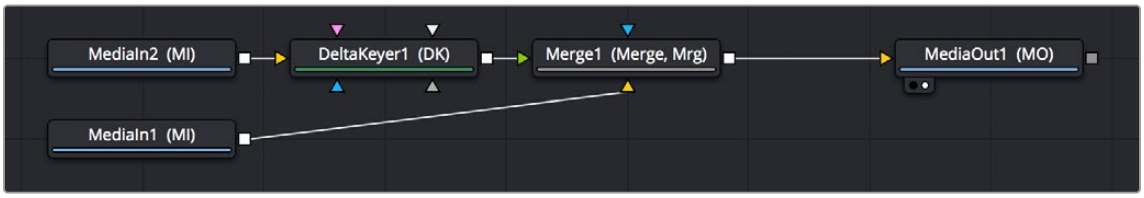
Under the hood, most nodes process all channels first, but afterward copy the input image to the output for channels that have been enabled. Modern workstations are so fast that this isn't usually noticeable, but there are some nodes where deselecting a channel actually causes that node to skip processing that channel entirely. Nodes that operate this way have a linked set of Red, Green, Blue, and Alpha buttons on another tab in the node. In these cases, the Common Control channel buttons are instanced to the channel buttons found elsewhere in the node.

Blur, Brightness/Contrast, Erode/Dilate, and Filter are examples of nodes that all have RGBA buttons in the main Controls tab of the Inspector, in addition to the Settings tab.

Adding Alpha Channels

Much of visual effects compositing has to do with placing a foreground subject over a background. Possibly the most fundamental method is through the use of an alpha or matte channel. If an alpha channel is not contained within the clip, you add one via keying or rotoscoping. While more specific methods are covered in detail in later chapters, here's an example of how this is handled within Fusion.

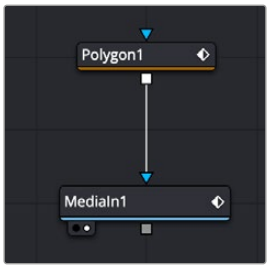
In the case of extracting an alpha matte from a green screen image, you typically connect the image's RGB output to the "Input" input of a Keyer node such as the Delta Keyer, and you then use the keyer's controls to extract the matte. The Keyer node automatically inserts the alpha channel that's generated alongside the RGB channels, so the output is automatically RGBA. Then, when you connect the keyer's output to a Merge node to composite it over another image, the Merge node automatically knows to use the embedded alpha channel coming into the foreground input to create the desired composite, as seen in the following screenshot.



A simple node tree for keying; note that only one connection links the DeltaKeyer to the Merge node

Rotoscoping, or manually drawing a mask shape using a Polygon or other Mask node is another technique used to create the matte channel. There are many ways to configure the node tree for this task, but the simplest setup is just to connect a Polygon or B-Spline mask node to the Effect Mask input of a MediaIn or Loader node.

TIP: When rotoscoping, it is best to leave the Mask node disconnected from the image while you draw the shape. This allows you to view the MediaIn node while drawing. Connect the Matte node once you have finished drawing the shape.

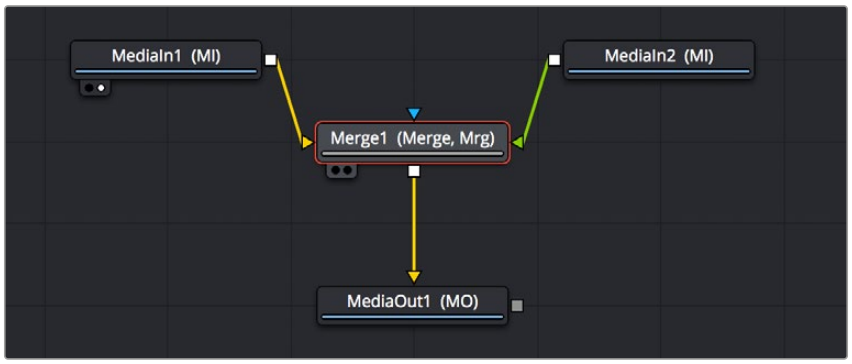


A simple rotoscoping setup using a Polygon node directly connected to the Effect Mask input of a MediaIn.

In both cases, you can see how the node tree's ability to carry a single channel or multiple channels of image data over a single connection line simplifies the compositing process.

How Channels Propagate During Compositing

Images are combined or composited together using the Merge node. The Merge node takes two RGBA inputs labeled "Foreground" (green) and "Background" (orange) and combines them into a single RGB output (or RGBA if both the foreground and background input images have alpha), where the foreground image is in front (or on top, depending on what you're working on), and the background image is, you guessed it, in back.



A simple Merge node composite

Auxiliary channels, on the other hand, are handled in a much more specific way. When you composite two image layers using the Merge node, auxiliary channels only propagate through the image that's connected to the background input. The rationale for this is that in most CGI composites, the background is most often the CG layer that contains auxiliary channels, and the foreground is a live-action green screen plate.

Since most compositions use multiple Merge nodes, it pays to be careful about how you connect the background and foreground inputs of each Merge node to make sure that the correct channels flow properly.

TIP: Merge nodes are also capable of combining the foreground and background inputs using Z-Depth channels using the "Perform Depth Merge" checkbox, in which case every pair of pixels are compared. Which one is in front depends on its Z-Depth and not the connected input.

Rearranging or Combining Channels

Last, but certainly not least, it's also possible to rearrange and re-combine channels in any way you need, using one of four different node operations. For example, you might want to combine the red channel from one image with the blue and green channels of a second image to create a completely different channel mix. Alternatively, you might want to take the alpha channel from one image and merge it with the alpha channel of a second image in different ways, adding, subtracting, or using other intersection operations to create a very specific blend of the two.

The following nodes are used to re-combine channels in different ways:

- **Channel Boolean:** This is a 3D node used to remap and modify channels of 3D materials using a variety of simple pre-defined math operations.
- **Channel Booleans:** Used to shuffle or rearrange RGBA and auxiliary channels within a single input image, or among two input images, to create a single output image. If you only connect a single image to this node, it must be connected to the background input to make sure everything works.
- **Copy Aux:** The Copy Aux node is used to remap channels between RGBA channels and auxiliary data channels in a single 2D image. The Copy Aux node is mostly a convenience node, as the copying can also be accomplished with more effort (and flexibility) using a Channel Booleans node.
- **Matte Control:** Designed to do any combination of the following: (a) re-combining mattes, masks, and alpha channels in various ways, (b) modifying alpha channels using dedicated matte controls, and (c) copying alpha channels into the RGB stream of the image connected to the background input in preparation for compositing. You can copy specific channels from the foreground input to the background input to use as an alpha channel, or you can attach masks to the garbage matte input to use as alpha channels as well.

Understanding Premultiplication

Now that you understand how to direct and recombine RGB images and alpha channels in Fusion, it's time to go more deeply into alpha channels to make sure you always combine RGB and alpha channels correctly for each operation you perform in your composite. This might seem simple, but small mistakes are easy to make and can result in unsightly artifacts. This is arguably one of the most confusing areas of visual effects compositing, so don't skip this section.

When alpha channel and RGB pixels are both contained within a media file, such as a 3D rendered animation that contains RGB and transparency, or a motion graphics movie file with transparency baked in, there are two different ways they might be combined, and it's important to know which is in use.

- **Unpremultipled (Straight):** An RGB image unaltered by the semi-transparency information in a fourth channel (alpha channel)
- **Premultiplied:** An RGB image that has each channel multiplied by its alpha channel before compositing.

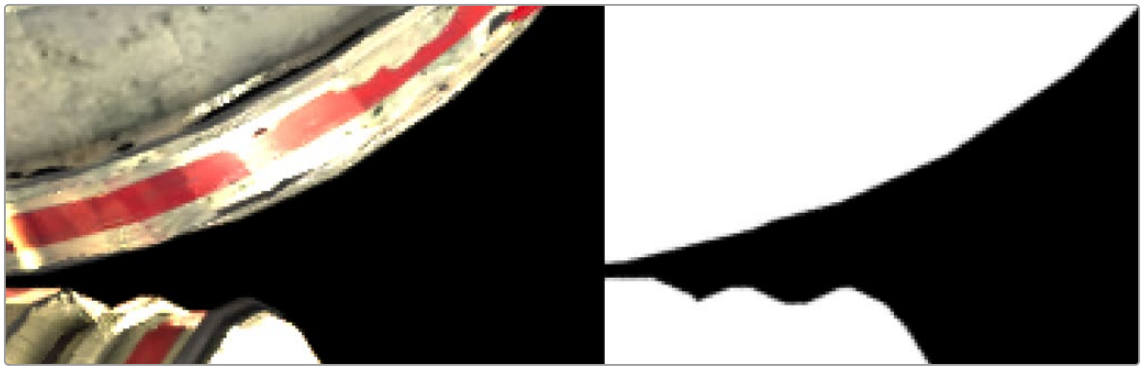
The term Premultiplied alpha is a term that has historically been used by editors, visual effects artists, and motion graphics designers, but it's imprecise. The alpha channel itself is not multiplied. The R, G, and B channels are multiplied by the alpha. In the end, the alpha channel stays the same, but the values contained in the R, G, and B channels are modified.

A RGB image (left) and its alpha channel (right)

Non-premultiplied images, sometimes called “straight” alpha channels, have RGB channels that are unaltered (not multiplied) by the alpha channel. The result is that the RGB image has no anti-aliased edges and no semi-transparency. It’s usually obvious where the RGB image ends and the alpha matte begins. The image below is an example of the ragged edges seen in the RGB channels when using a non-premultiplied alpha channel. But notice the smooth semi-transparent edges found in the alpha.

A detailed view of a non-premultiplied RGB image (left) and its alpha channel (right)

A premultiplied alpha channel means the RGB pixels are multiplied by the alpha channel. This method guarantees that the RGB image pixels include semi-transparency where needed, like anti-aliased edges. Most computer-generated images are premultiplied for convenience, because they’re easier to review smoothly without actually being placed inside of a composite.



A detailed view of a premultiplied image (left) and its alpha channel (right)

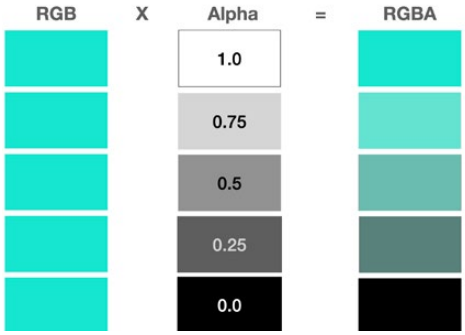
What does this mean for compositing? The edges of a premultiplied image look much smoother, making them the preferred choice when compositing foreground over the background in a merge. Consequently, all alpha channels are made to be premultiplied in compositing operations if they weren’t already.

On the other hand, it is always preferred to color correct a non-premultiplied RGBA image, because you don’t want to alter the pixel values of an image after the RGB channels have been multiplied by the alpha channel.

If you think of this from a mathematical perspective:

- **RGB pixel value x 0 = 0:** The black transparent areas of an alpha channel have a pixel value of 0. When you take the value of an RGB pixel and multiply it by 0 (n x 0 = 0) then by the laws of multiplication, the RGB value becomes 0, or fully transparent.
- **RGB Pixel value x 1 = RGB Pixel:** The solid or opaque white areas have a value of 1.0. When you take the value of an RGB pixel and multiply it by 1 (n x 1 = n), then the RGB value stays the same, fully opaque.

- **RGB Pixel value x 0.3 = A different color:** Along the edges of an alpha channel are gray pixels, indicating semi-transparency. These semi-transparent pixels have a value falling somewhere between 1.0 and 0.0. To apply the alpha channel's anti-aliased edges to the RGB channels, you multiply the pixel values. The multiplication process mixes some percentage of the transparent pixels (black) with the RGB pixels. Although this is desired to get good anti-aliased edges, you can not color correct the image because it alters the smooth semi-transparency you created once it is done.



RGB pixels are multiplied by varying degrees of transparency and the result is a different RGB value.

The Rules of Premultiplication

Following from the information presented above, when you're compositing multiple images together and one or more has a built-in alpha channel, you want to make sure you follow these general rules to avoid problems:

- Always use premultiplied images with a Merge node.
- Only color-correct images that are not premultiplied.
- Always filter and transform images that are premultiplied.
- Never double premultiply an image.

Premultiplication and the Merge Node

The foreground input of a Merge node expects a premultiplied RGBA image. It is an additive merge, meaning the semi-transparent areas of the foreground are added over the background. However, if the image is not premultiplied, the pixels that should be transparent are still added, which typically results in an unwanted bright fringe around the edges of your foreground subject.

If you are compositing with a non-premultiplied alpha, you can fix these bright edges by changing the Merge to perform a Subtractive merge in the Inspector.

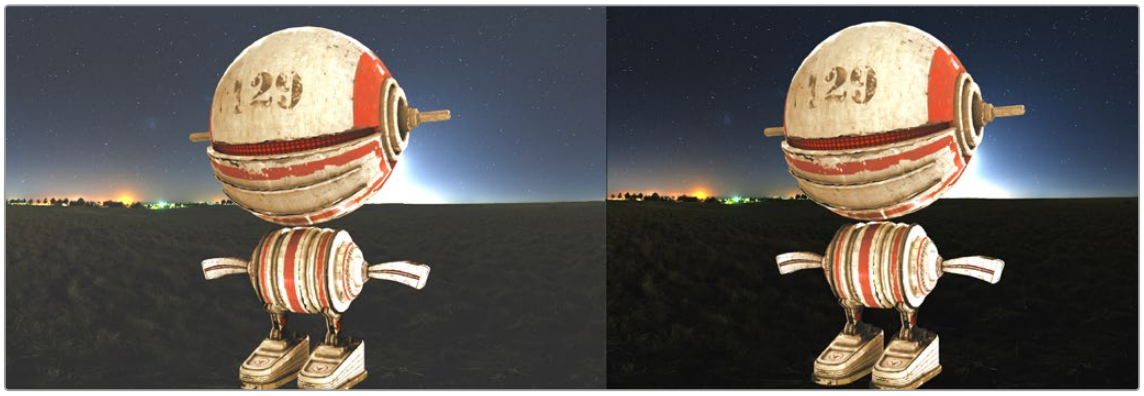
Nonpremultiplied edges in an additive Merge (left) and premultiplied edges in an additive Merge (right)

TIP: When an RGB image and a Mask node are combined using, for instance, a Matte Control node, if the RGB image is not multiplied by the mask in the Matte control, the checkerboard background pattern in the viewer will appear only semi-transparent when it should be fully transparent.

Color Correcting Premultiplied RGBA Images

When you premultiply an image, you tie any changes made to the brightness of the RGB pixels to the alpha channel pixels as well. If, for example, you raise the brightness of the foreground image in any way, you'll also raise the brightness of the alpha channel, which will likely not be desirable as this will change the transparency the alpha channel creates. The visible result of this when viewing the Merge node is that the entire background will become brighter (or darker if you'd lowered the RGB brightness) based on your color adjustment.

For this reason, the rule is always to divide the semi-transparent pixels before performing any color correction on an image with an alpha channel. You can do this turning on the Pre-Divide/Post Multiply checkbox in any node that performs color correction. Alternatively, you can use the Alpha Divide and Alpha Multiply nodes to do the same thing. These methods are covered in more detail later in this chapter.



Color correcting a premultiplied foreground incorrectly alters the background (left).
Color correcting a nonpremultiplied foreground works correctly (right).

Double Premultiplied RGBA Means Double Trouble

A common mistake made by many artists is to over-compensate for premultiplication. As important as it is to premultiply the alpha before compositing in a Merge node, it's just as important not to double premultiply the alpha. Performing a premultiply operation two times in a row can create a darken halo effect around your images. You are effectively multiplying by the gray semi-transparent pixels twice; this is not optimal.

Double premultiplied image displays dark edges (left);
premultiplied image with correct edges (right)

Premultiplied Alpha Channels and Filtering

When dealing with filtering, the state of the RGBA channels shouldn't matter for most composites. However, an exception might be if the filter algorithm you choose includes color modification. For instance, if a filter attempts to simulate a defocus by blooming highlights like a real light source, that filter might over-brighten pixels near a transparent edge, which will result in some manner of artifact when that image is composited.

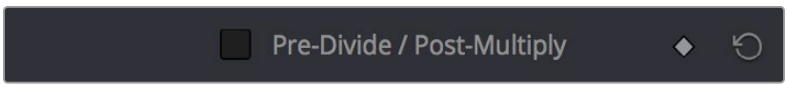
Alpha Channel Status in MediaIn and Loader Nodes

When using a Loader node to add a clip to the composite, the Import tab in the Inspector includes a group of checkboxes that let you determine how an embedded alpha channel is handled. There are checkboxes to make the alpha channel solid (ignore transparency), to Invert the alpha channel, and to Post-Multiply the RGB channels with the alpha channel, should that be necessary.

When using a MediaIn node, you can modify how an embedded alpha channel is interpreted using the Clip attributes window. The Clips Attributes window includes an Alpha Mode menu setting to choose if the alpha channel is ignored, treated as premultiplied, inverted, or treated as nonpremultiplied (straight).

Controlling Premultiplication in Color Correction Nodes

Most nodes that require you to explicitly deal with the state of premultiplication of an RGBA image have a "Pre-Divide, Post-Multiply" checkbox. This includes simple color correction nodes such as Brightness Contrast and Color Curves, as well as the Color Correct node, which has the "Pre-Divide/Post-Multiply" checkbox in the Options panel of its Inspector settings.

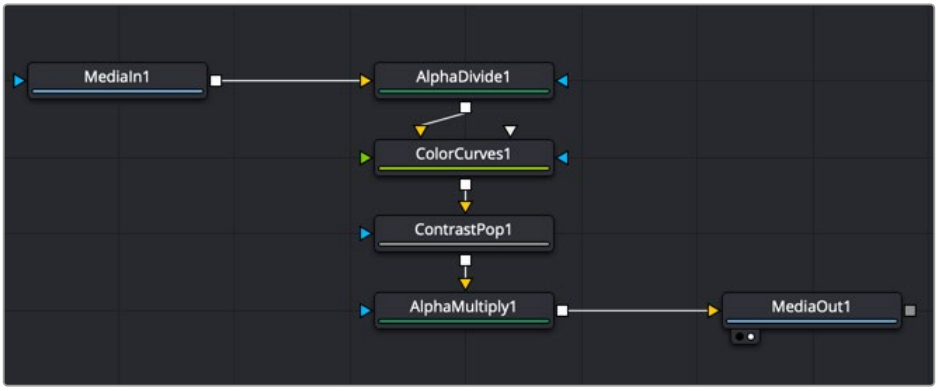


The Pre-Divide/Post-Multiply checkbox of t
he Color Curves node, seen in the Inspector

This checkbox allows you to connect an RGBA premultiplied image to the node and perform a color correction operation. It takes the RGBA image input, performs a divide operation to remove the semi-transparency and then performs a multiplication operation before outputting the color corrected image. This way, the color correction is done using a nonpremultiplied image but the resulting output is a Merge-friendly premultiplied image.

Controlling Premultiplication With Alpha Divide and Alpha Multiply

The Alpha Divide and Alpha Multiply nodes, found in the Matte category of the Effects Library, are available when multiple operations in a row expect a "straight" alpha channel. Instead of performing repetitive Pre-Divide/Post Multiply operations on each node, you can use these two nodes to book-end the other nodes. Simply add the Alpha Divide node when you want the RGBA image data not to be premultiplied, and add the Alpha Multiply node when you want the image data to be premultiplied again. For example, if you're using third-party OFX nodes that make color adjustments, you may need to control premultiplication before and after such an adjustment manually.



A node tree with explicit Alpha Divide and Alpha Multiply nodes

Multi Channel Compositing

If you've gone to the cinema and seen any recent superhero movie, you will have seen the results of sophisticated 3D rendering, combined with a whole lot of compositing. 3D applications can render very realistic images but the time it takes to render each frame of those realistic images can be measured in hours, not minutes. Any change to the 3D images, even relatively simple operations like color adjustments, changes to focus, filtering, or additional masking, means these images will need to be re-rendered entirely, which means you waiting, often many times over. For efficiencies sake, it's much faster to make iterative changes that can be accomplished with 2D image processing operations in Fusion instead.

To have the flexibility you need to make common changes to 3D images after-the-fact, the various attributes that make up the 3D scene are separated and rendered as different image sequences, often referred to as render passes. For example, render passes are often created for attributes like raw color, shadows, and reflections, that can be recombined as a 2D composite to produce the final result. Having different attributes rendered into different image sequences gives you a significant amount of flexibility, since now each image attribute can be color corrected, blurred, or further processed independently of the other attributes of the image, with fast-processing operations in Fusion.

The most common render passes that are typically generated come from the RGBA channels of the 3D scene. These are collectively called beauty passes and can consist of attributes like color, shadows, lighting, reflections, environment, and others.

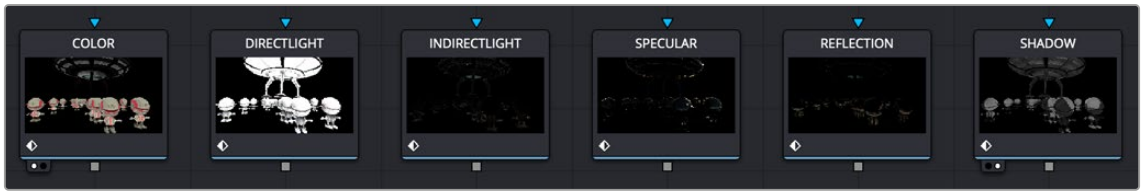
Render passes can also contain non-RGB data. Different effects applications have different names for these passes, such as Data Channels, or AOVs (Arbitrary Output Variables). In Fusion, these channels are called Auxiliary Channels, and they contain 3D data such as Depth, Normals, Motion Vectors, and UV Coordinates (to name just a few).

When compositing a 3D render consisting of multiple render passes, the beauty passes are handled using one technique, and the Auxiliary Channels are handled with another. Since Fusion nodes carry RGBA channels by default, we'll cover beauty passes first, and then explain how to work with Auxiliary Channels later in this chapter.

Compositing with Beauty Passes

Each attribute of a beauty pass can be rendered into individual image sequences, so you end up with a series of numbered images, one for the diffuse pass, one for the reflection pass, another of the shadow pass, and so on. Alternatively, all the passes can be contained in a multi-part EXR image sequence. Multi-part EXRs benefit from requiring a bit less file management, but the passes are handled similarly in Fusion using either method.

A single MediaIn or Loader node only handles a single beauty pass since only one set of RGBA channels gets output per node. Setting up your composite in Fusion requires you to use a separate MediaIn or Loader node for each pass.

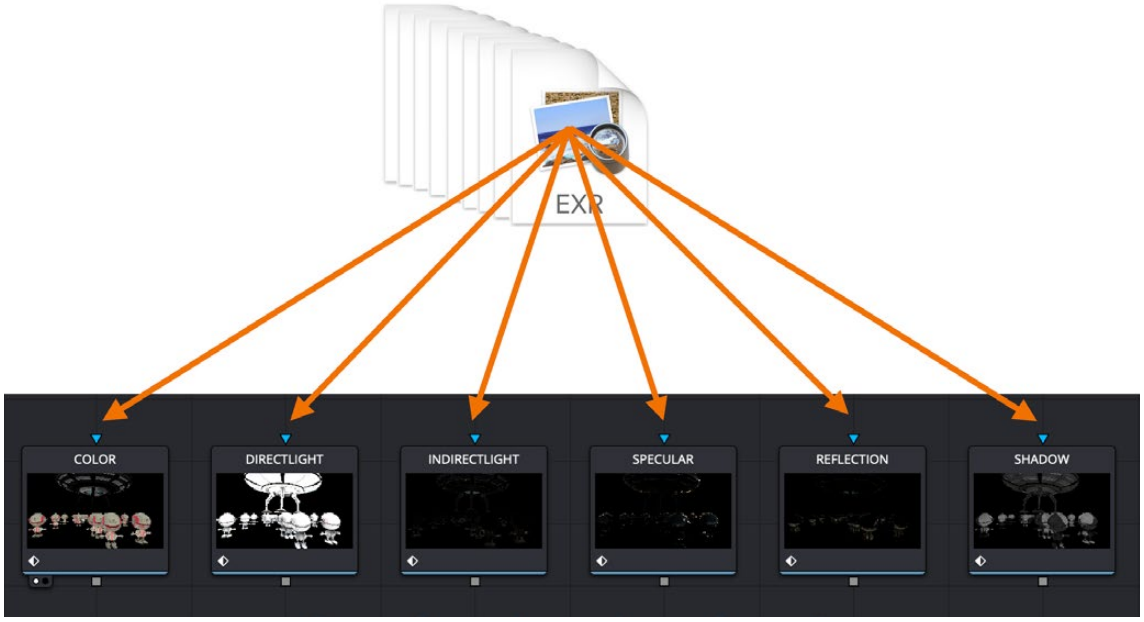


A MediaIn or Loader node is needed for each beauty pass.

TIP: The EXR format in Fusion is optimized when multiple Loaders are used to read the same EXR file. The file is only loaded once to access all the channels.

Beauty Pass Setup

Separated components of 3D renders, such as diffusion, shadows, or reflections, can be rendered individually using RGB channels. If you're provided with individual image sequences for each image component, they'll import and open in Fusion and can be displayed in the viewer, just like any other clip. If you're using multi-part EXR files, then you essentially have multiple RGB images contained within a single file. However, the MediaIn and Loader nodes can only make use of one set of RGB channels at a time. If you want to composite multiple RGB beauty passes together with one another, you must use multiple MediaIn or Loader nodes that point to the same image sequence but are assigned to different passes contained in the EXR file.

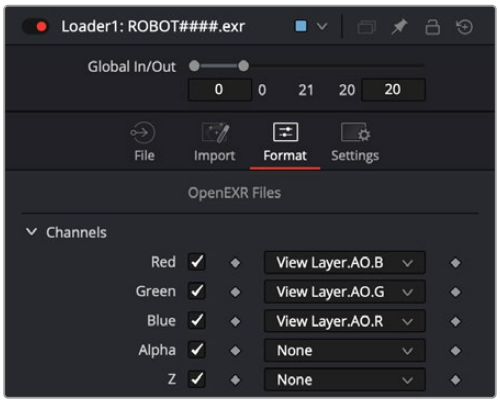


Multiple Loader or MediaIn nodes connect to a multi-part EXR image sequence

TIP: It is wise to rename each Loader or MediaIn to represent the beauty pass it contains.

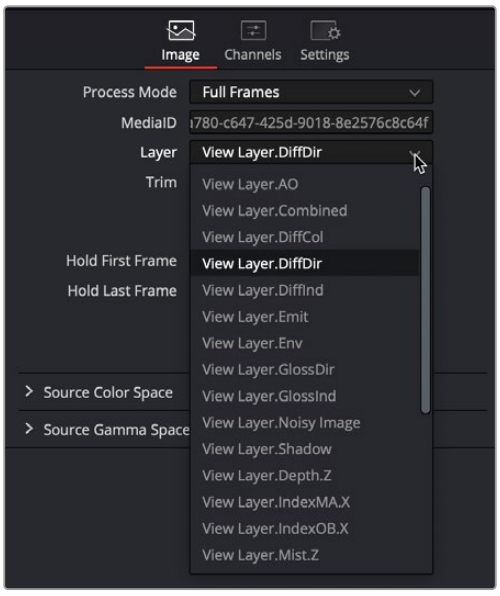
Mapping Each Set of Beauty Passes to a Particular Node

Depending on whether you're using a MediaIn node or a Loader node, beauty passes can be mapped to RGB channels using either the Image tab, the Channels tab, or the Format tab in the Inspector. When using a Loader node, the Loader's Format tab is used.



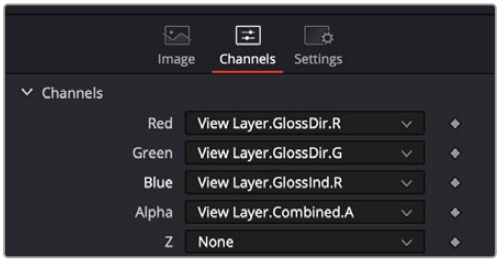
Beauty passes mapped to red, green, and blue channels in a Loader node

The MediaIn's Image tab includes a Layer menu. Any pass included in a multi-part EXR image sequence can be selected from this menu and automatically assigned to the RGBA channels.



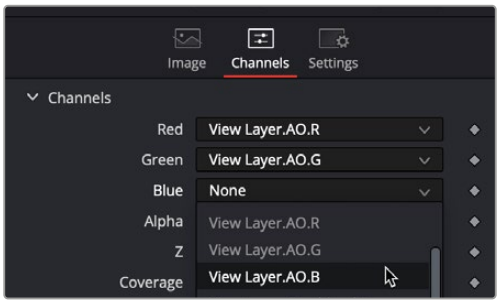
The Layer menu in a MediaIn node showing headings for combined channel passes

In most cases, the menu shows the combined channel passes, meaning the individual red, green, blue, and alpha channels cannot be selected. Because the alpha channel is not included in many beauty passes, you sometimes need to borrow the alpha channel from a different beauty pass. For this reason, it's often better to use the Channels tab for mapping the individual channels of a beauty pass to the channels of the MediaIn node.



The MediaIn node's Channels tab or the Loader's Format tab provides access to individual channels.

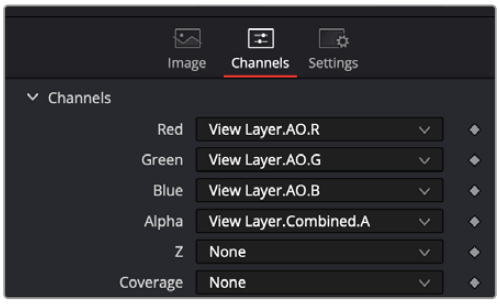
The MediaIn node in DaVinci Resolve and the Format tab in a Loader node include the same channel mapping functionality. The Channels tab and the Format tab include individual RGBA menus at the top of the tab. You can use these menus to map the RGBA channels from any pass contained in the multi-part EXR. For instance, if you want to map the Ambient Occlusion pass to the RGB channels, choose AO. R (Red) from the Red channel menu, AO. G (Green) from the Green channel menu, and AO. B (Blue) from the Blue channel menu.



A pass' individual channels mapped to the red, green, blue, and alpha channels in the MediaIn node

TIP: Different 3D applications will label beauty passes in different ways. For instance, the name for an Ambient Occlusion beauty pass may be AO, AM_OCC, or some other abbreviation.

The Ambient Occlusion beauty pass does not include an alpha channel. To composite it, you can reuse the alpha channel pass from another beauty pass. In the image below, the alpha channel is mapped using the combined render pass' alpha channel.

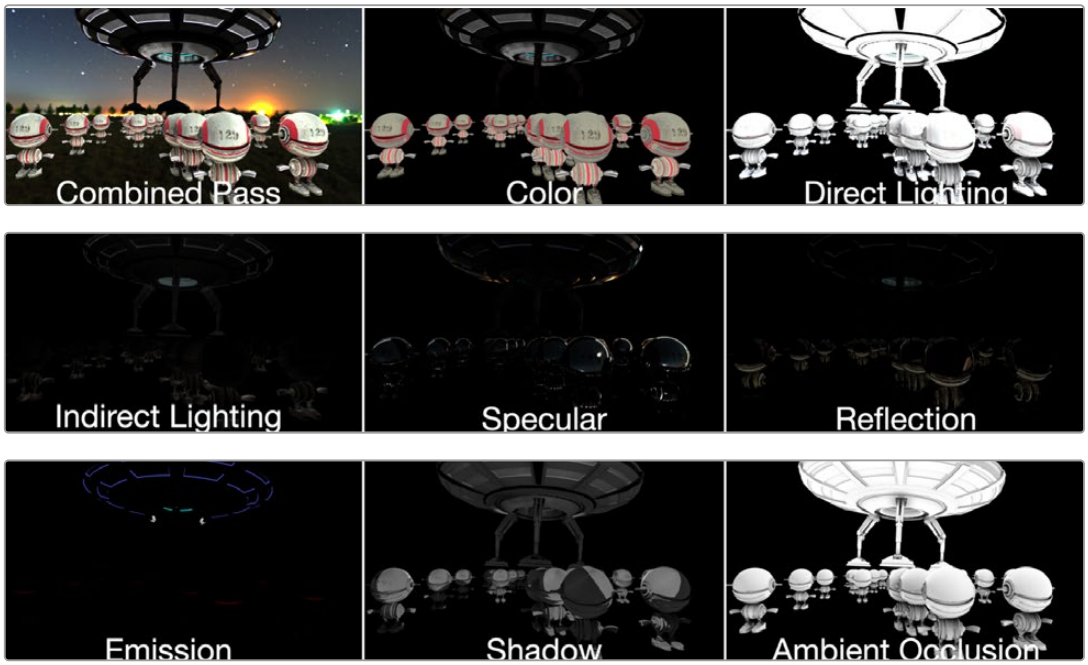


The alpha channel from a different beauty pass combined with the Ambient Occlusion pass

TIP: When using the Format tab in the Loader node, the checkbox next to each channel needs to be turned on for the corresponding channel to become available in the node's output.

Compositing Multiple Beauty Passes in the Node Editor

Once all of your passes are brought in and mapped to RGBA channels, you'll end up with a series of MediaIn or Loader nodes. How many MediaIn and Loader nodes is really up to your workflow. There is no predefined number of passes you will use. Every studio decides for themselves the standard set. However, there are common render passes that are involved in most composites. The following is a list of commonly used render passes and their generic names.



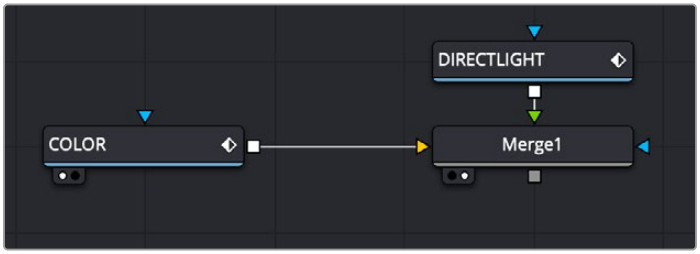
Commonly used beauty passes, compared

Compositing multiple beauty passes into a single output image is relatively straight forward. 3D rendering applications typically output linear gamma, so no Gamut or other color space conversion nodes are required if you're keeping the image in a linear color space for ease of compositing.

The basic compositing is accomplished with either a Merge node or a Channels Booleans node. Both allow for additive combining of render passes. There's no strict requirement for compositing each pass in any particular way, although in most situations a simple additive composite should work just fine.

To begin compositing render passes using a Merge node:

1. Connect a Color pass to the background input of a Merge node.
2. Connect a Direct Lighting pass to the foreground input.
3. Adjust the Alpha gain and Blend parameters to get the look you desire.

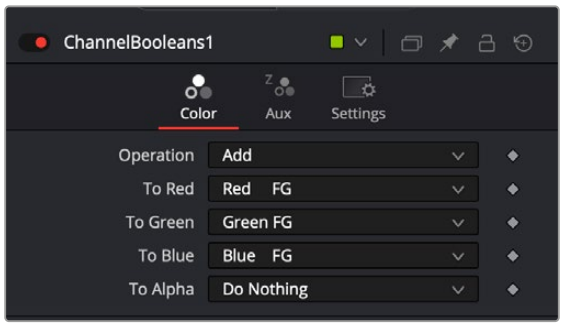


Compositing beauty passes starts by connecting a background and foreground into a Merge node

If you prefer, you can use a Channels Booleans node to make the same composite. In this case, there is no technical difference between the two nodes.

To composite render passes with a Channels Booleans node:

1. Connect a Color pass to the background input of a Channels Booleans node.
2. Connect a Direct Light pass to the foreground input.
3. Choose Add from the Operations menu
4. Choose Do Nothing from the Alpha To menu.



Channel Booleans is set to Add to combine the foreground input and the background input.

One of the exceptions to the steps above are Shadow passes, such as Ambient Occlusion. In that case, a *multiply* Apply mode is usually employed.

To composite an Ambient Occlusion render pass using a Merge node:

1. Connect last of a sequence of Merge node render passes to the background input of a Merge node.
2. Connect an Ambient Occlusion pass to the foreground input.
3. Choose Multiply from the Apply Mode menu.
4. Adjust the Gain and Blend parameters to get the look you desire.

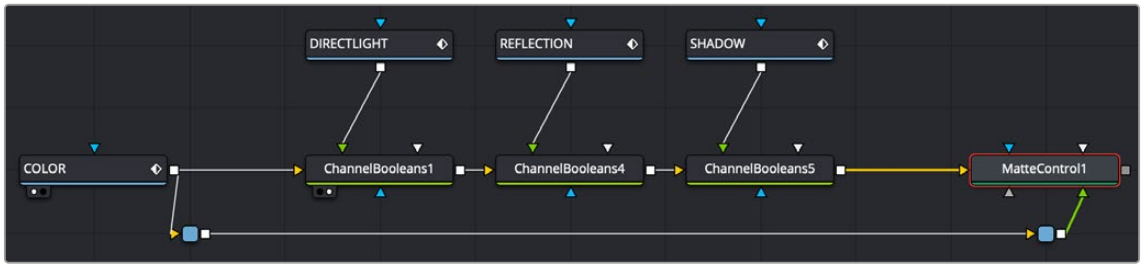
As straightforward as this sounds, compositing using a recipe doesn't always work for every shot. When using different images, you may need to experiment with varying techniques of compositing for the best results.

Embedding Alpha into Beauty Passes

Alpha channels are not included in all beauty passes. If your shot requires that you composite your assembled beauty passes over live-action or some other background, it may be up to you to add the alpha channel from a pass that does include it.

To add an alpha channel into your assembled beauty pass composite, do the following:

1. Connect the last Merge or Channel Booleans output into the background input of a Matte Control node.
2. Connect the render pass that contains the alpha into the green Foreground input of the Matte Control node.
3. In the Matte Control's Inspector, choose Combine Alpha from the Combine menu.
4. Choose Copy from the Combine Op menu.



An alpha channel from the color pass added back into the completed beauty pass node tree

TIP: Alpha channels from 3D renderings are typically premultiplied. That being the case, be sure to turn on the Pre Divide/Post Multiply checkbox on any node that performs color correction. If using more than one node in a row to perform color correction, use the Alpha Divide and Alpha Mult nodes instead.

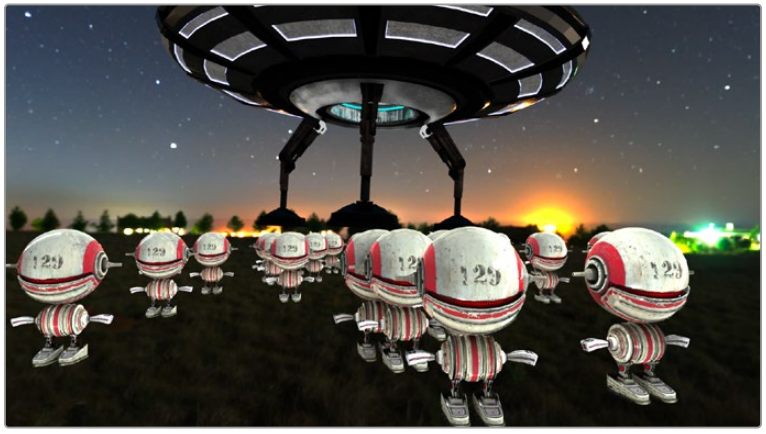
Working with Auxiliary Channels

Auxiliary channels are not RGB images; they are a family of special-purpose 3D image data that typically describes position, orientation, and object information for use in 2D composites. For example, Z-Depth channels describe the depth of each pixel of an image along a Z-axis, while an XYZ Normals channel describes the orientation (facing up, facing down, facing to the left or right) of every pixel in an image.

Similar to the use of multiple beauty passes, one of the most common reasons to use auxiliary data is to eliminate the need to re-render computationally expensive 3D imagery, by enabling even more aspects of rendered images to be manipulated after-the-fact. 3D rendering is computationally expensive and time-consuming, so outputting descriptive information about a 3D image allows sophisticated alterations to occur in 2D compositing, which is faster to perform and adjust.

There are two ways of obtaining auxiliary channel data:

- First, auxiliary data may be embedded within a clip rendered from a 3D application, most often using the EXR file format. In this case, it's best to consult your 3D application's documentation to determine which auxiliary channels can be generated and output.
- You may also obtain auxiliary channel data by generating it within Fusion, via 3D operations output by the Renderer 3D node, by the Optical Flow node, or by the Disparity node.

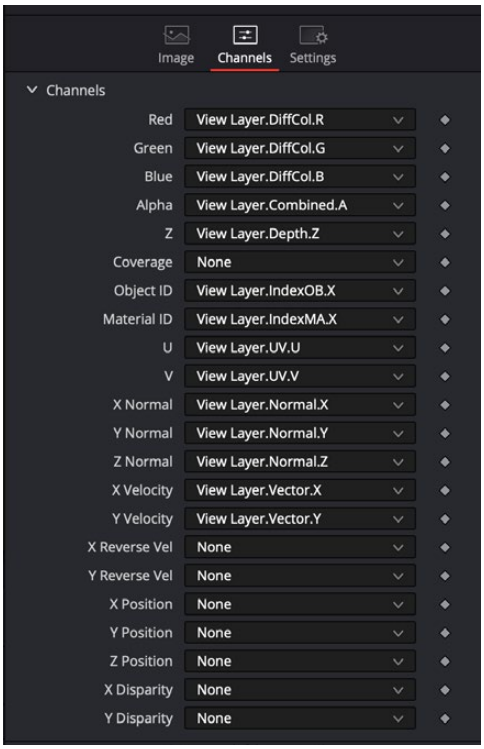


An RGBA 3D rendered scene that also contains auxiliary channels

TIP: When trying to locate information about auxiliary channels in other software, some 3D applications refer to auxiliary channels as Arbitrary Output Variables (AOVs), render elements, or secondaries.

Aux Channel Setup

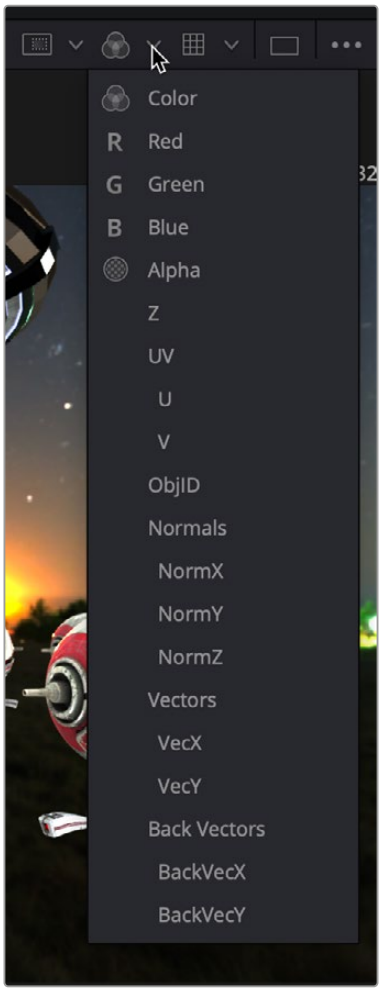
When using a MediaIn or Loader node that links to a multi-part EXR file with auxiliary channel data, the Inspector's Channels or Format tab includes a pre-defined set of auxiliary channels for mapping purposes. Each pre-defined channel includes a menu that displays every attribute included in a multi-part EXR. From the menu, you select the render pass that should be assigned to the corresponding channel. As described earlier, RGB beauty passes like diffuse, shadow, and reflection are mapped to the red, green, and blue channels. Auxiliary passes include preset mappings.



A multi-part EXR file with embedded render passes mapped to their aux channels in a MediaIn node

Displaying Channels in the Viewer

Once you map an auxiliary channel in the Inspector, you can display the data as an RGB image in the viewer. Clicking the drop down Color menu at the top of the viewer reveals a list of every active auxiliary channel for the currently viewed node.



Select an aux channel from the color drop-down menu to display the aux channel in the viewer

TIP: The Color Inspector SubView can be used to read numerical values from all of the channels.

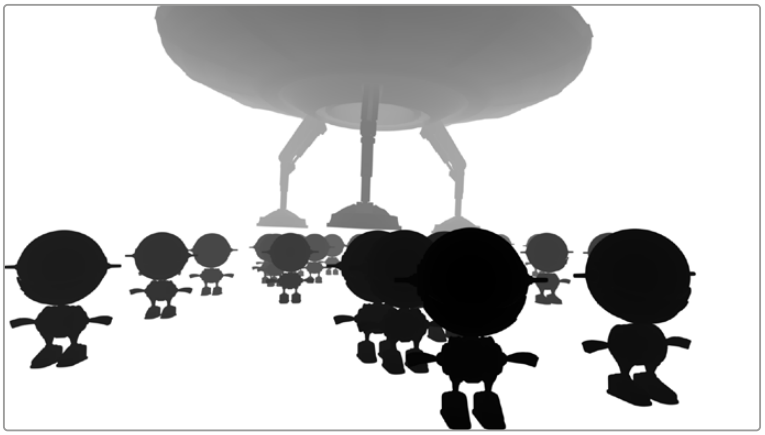
Auxiliary Channels Explained

Fusion is capable of using auxiliary channels to perform depth-based compositing, to create masks and mattes based on Object or Material IDs, and for texture replacements. Nodes that work with auxiliary channel information have been specifically developed to work with this data. The auxiliary channels that are supported in Fusion are described below.

Z-Depth

Each pixel in a Z-Depth channel contains a value that represents the relative depth of that pixel in the scene. In the case of overlapping objects in a model, most 3D applications take the depth value from the object closest to the camera when two objects are present within the same pixel since the closest object typically obscures the farther object.

When present, Z-Depth can be used to perform depth merging using the Merge node or to control simulated depth-of-field blurring using the Depth Blur node.

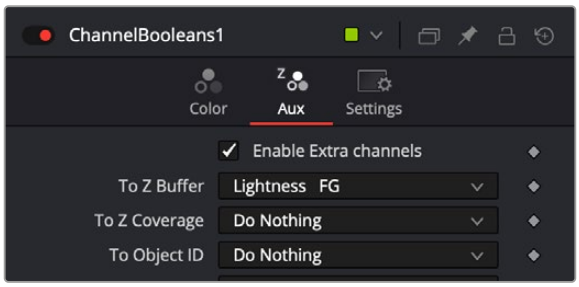


The rendered Z-Depth channel for the previous RGBA image

For this example, we'll examine the case where the Z-Depth channel is provided as a separate file. The Z- channel can often be rendered as an RGB image. You'll need to combine the beauty and Z pass using a Channel Booleans node. When the Z pass is rendered as an image in the RGB channels, the Channels Booleans node is used to re-shuffle the Lightness of the foreground RGB channel into the Z-channel.

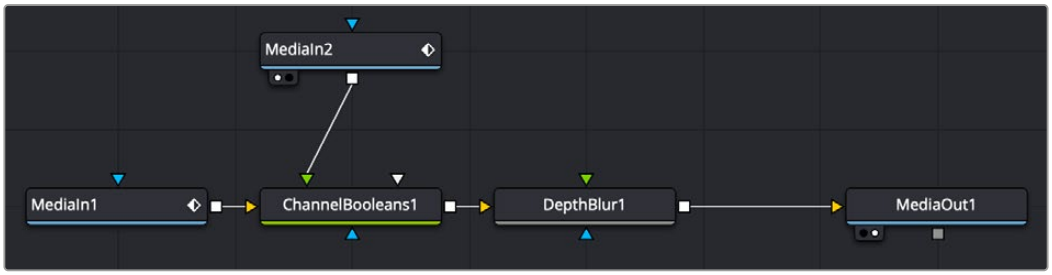
To combine the Z-pass and beauty pass:

1. Connect the MediaIn node containing the beauty pass to the background input of the Channel Booleans node.
2. Connect the MediaIn node containing the Z-Depth pass to the green foreground input of the Channel Booleans node.
3. Select the Channel Booleans node, and use the Inspector to set the To Red, To Green, To Blue, and To Alpha menus to Do Nothing.
4. Select the Aux tab, and set the To Z Buffer menu to Lightness FG.
5. Connect the output of the Channels Booleans node into the Depth Blur node.



The Aux tab configured to shuffle the Foreground Lightness to the Z-Depth channel.

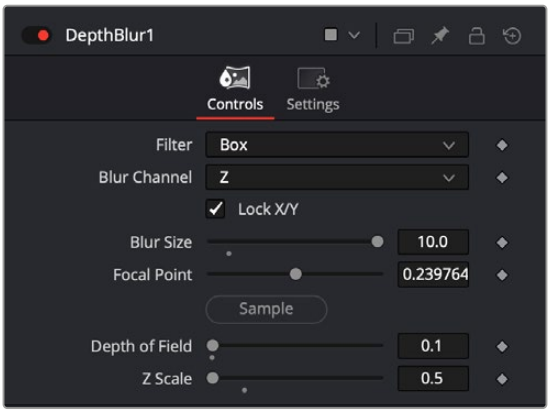
The Depth Blur node is one of the nodes that take advantage of a Z-channel in order to create blurry depth-of-field simulations. To set this up, the output of the MediaIn node connects to the background input on the Depth Blur.



The Depth Blur uses the Z-channel that is enabled in the Channel Booleans node.

The Depth Blur's controls in the Inspector are very dependent on the type of image you're using. It can be easier to begin by adjusting the controls in the Inspector to some better defaults. Start by increasing the Blur Size to 10. This will make it easier to see even the smallest of changes. Next, instead of using the Focal Point, you should pick a focal point in the image by dragging the Sample button into the viewer and selecting a pixel that determines the part of the picture to keep in focus.

The final setup steps are to lower Z Scale to somewhere around 0.2 (if you're using a floating-point image), and leave the Depth of Field alone for now. This should show you some blurring in the image.



Start by improving the defaults if your image is 16- or 32-bit floating point.

Once you see these experimental results, you can return to each parameter and refine it as needed to achieve the actual look you want.

An image using a Z-Depth channel for blurring

TIP: Z-Depth channels often contain negative values. If this causes problems, you can choose Normalize Color Range from the viewer's Options menu to apply a normalization to the viewer, keeping the image within a range from 0 to 1.

Z-Coverage

The Z-Coverage channel is a somewhat extinct render pass in most 3D applications. It was a way of restoring antialiasing to rendered color masks and Z-Depth passes. It indicated pixels in the Z-Depth that contained two objects. The value was used to indicate, as a percentage, how transparent that pixel was in the final depth composite. It can still be used today if you are rendering files from one of the few applications that can produce them.

TIP: The wide adoption of an open-source matte creation technology called Cryptomatte, has somewhat superseded mattes created from Coverage, Background, Object ID, and Material ID passes.

Background RGBA

This channel is a somewhat extinct render pass in most 3D applications. It contained the color values from the objects behind the pixels described in the Z coverage.

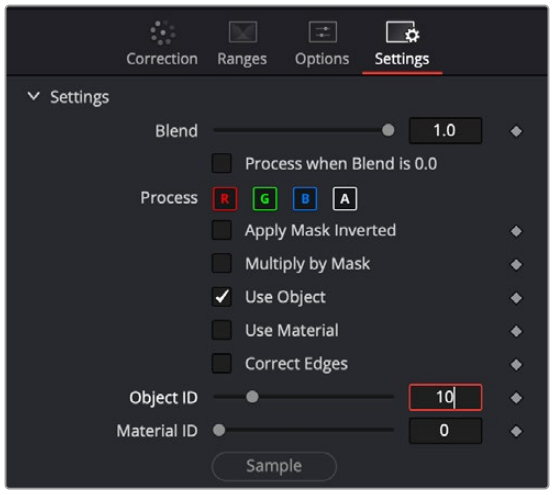
Object ID

Most 3D applications are capable of assigning ID values to objects in a scene. Each pixel in the Object ID channel will be identified by that ID number, allowing for the creation of masks.

If you want to use an Object ID in a comp, like all aux channels you must map the Object ID pass to the Object ID channel in the MediaIn or Loader Node.

To use an ObjectID pass, do the following:

1. In the MediaIn or Loader node, use the Channels or Format tab to map the Object ID pass to the Object ID aux channel.
2. In whatever node you want to have affected by the ObjectID matte, select the Settings tab, turn on the Object ID checkbox, and select the ID number assigned to the object.



The common Settings tab on most nodes contains ObjectID controls.

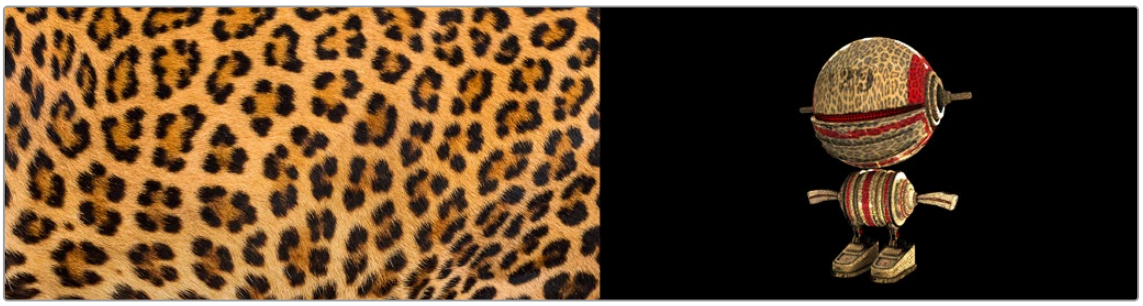
Material ID

Most 3D applications are capable of assigning ID values to materials in a scene. Each pixel in the Material ID channel will be identified by that ID number, allowing for masks based on materials.

You can set up Material IDs using the Settings tab, similarly to how ObjectIDs are set.

UV Texture

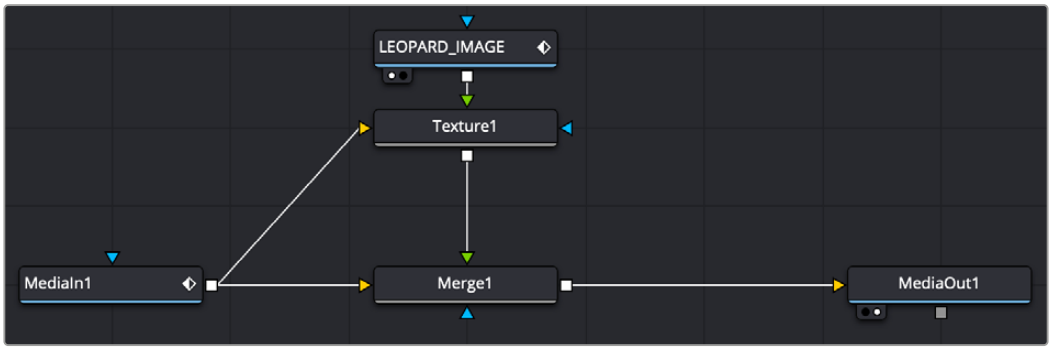
The UV Texture channels contain information about how pixels in an image map to texture coordinates. This is used to retexture an object in a 2D image. For instance, If you want to apply a logo onto a rendered object, you can use the UV aux channel with the Texture node.



Texture (left) applied to 2D image (right) using UV channels and texture node.

To use a UV pass, do the following:

1. In the MediaIn or Loader node, use the Channels or Format tab to map the U and V pass to the U and V aux channels.
2. Connect the output of the MediaIn or Loader node to the background input of the Texture node.
3. Connect the texture image you want to use to the foreground input of the Texture node.
4. If you want to combine the original texture with the new texture, use a merge with the background input from the original image and the foreground input from the Texture node.
5. Adjust Merge's Apply mode, Alpha Gain, and blend to get the desired mix of the two textures.

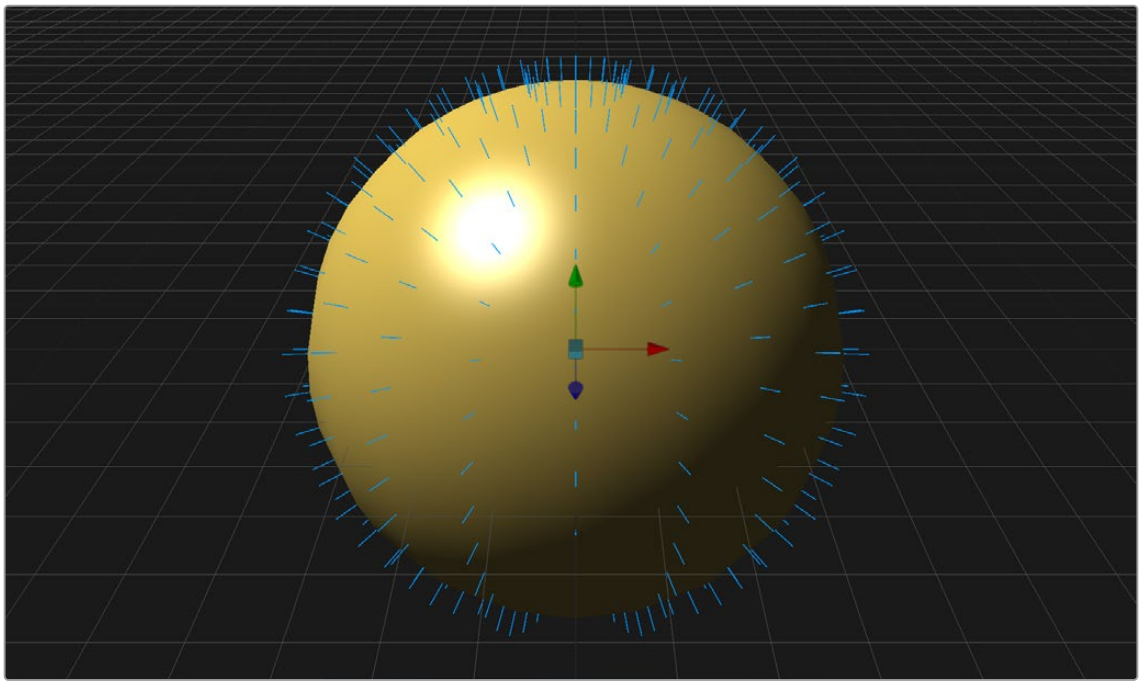


UV channels from a MediaIn node used in a Texture node and merged over the original image

TIP: If you are using a separate UV render pass with the UV data in the RGB channels, map red to U and green to V in a Channel Booleans node.

X, Y, and Z Normals

The X, Y, and Z Normal channels contain information about each pixel's orientation (the direction it faces) in 3D space. The normals are often displayed as lines coming out from your object perpendicular to the surface, letting you visualize the relationship between the surface and camera.



The Normals display the direction of the surface.

The Normals X, Y, and Z channels are often used with a Shader node to perform relighting adjustments on a 2D rendered image.

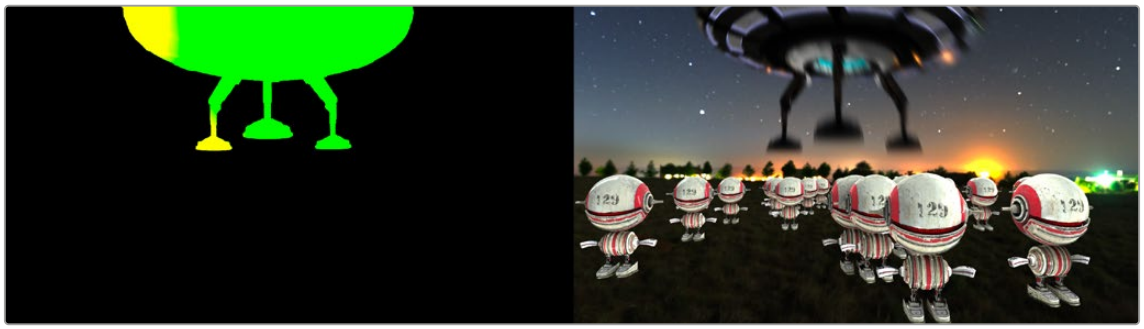
To setup a Shader node to use XYZ Normals, do the following:

1. In the MediaIn or Loader node, use the Channels or Format tab to map the individual X, Y, and Z Normals pass to the X Normal, Y Normal, and Z Normal channels.
2. Connect the output of the MediaIn or Loader node to the background input of the Shader node.
3. Optionally, connect a floating-point EXR image to be used as a reflection image to the Shader node's reflection input.
4. Adjust the Shader controls to perform relighting.

Original 2D image (left) and Normals used for relighting (right)

XY Vector and XY BackVector

The Vector channels indicates the pixel's motion from frame to frame. It can be used to apply motion blur to an image or to generate optical flow analysis for retiming. The XY Vector points to the next frame, while the XY BackVector points to the previous frame.

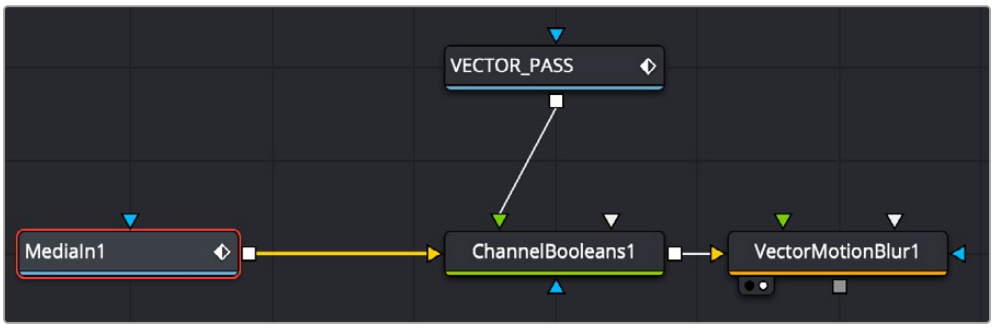


XY Vector pass (left) used with Vector Motion Blur to generate motion blur on spaceship (right)

Often the vector pass will be rendered in a separate pass as an RGB image. The X and Y vector data is located in the R and G channels. In order to place them in the vector channels, you can use a Channel Booleans node.

To use a Motion Vector pass to create motion blur, do the following:

1. Add a MediaIn or Loader node for the image and the Vector render pass.
2. Connect the output of the image into the background of a Channel Booleans node.
3. Connect the output of the Vector render pass to the Channel Boolean's foreground
4. In the Channel Booleans inspector, set the To Red, To Green, To Blue, and To Alpha all to Do Nothing.
5. Select to the Aux tab.
6. Turn on Enable Extra Channels.
7. Set the To X Vector drop-down menu to Red FG, and then set the To Y Vector drop-down menu to Green FG.
8. Connect the Channel Booleans node's output to the yellow background input on a Vector Motion Blur node.

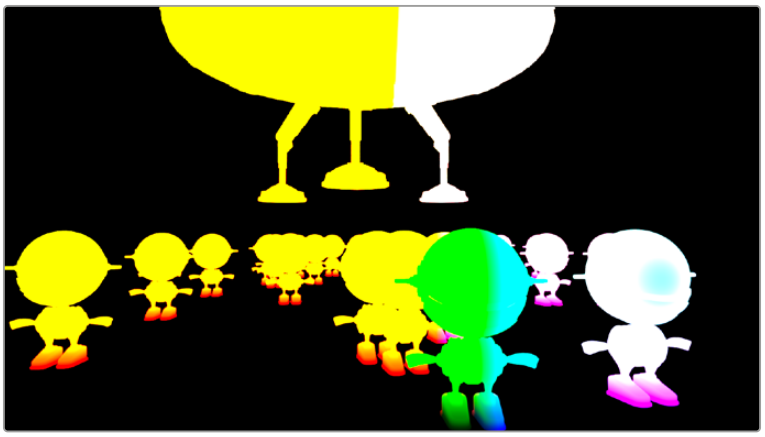


The Vector render pass is combined with the beauty image using the Channels Booleans node, which then feeds the Vector Motion Blur node.

World Position

World Position Pass (WPP) is an auxiliary channel, sometimes referred to as Point Position, XYZ pass, or WPP. It's used to represent each pixel's 3D (XYZ) position as an RGB color value. The result is data that can be viewed as a very colorful RGB image. Like Z-Depth, this can be used for compositing via depth. However, it can also be used for masking based on 3D position, regardless of camera transforms.

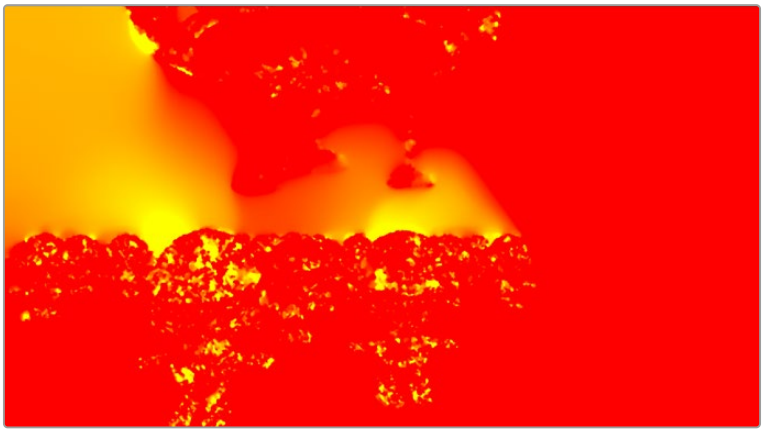
The colors correspond to a pixel's position in 3D, so if a pixel sits at 0/0/0 in a 3D scene, the resulting pixel's will have an RGB value of 0/0/0 or black. If a pixel sits at 1/0/0 in the 3D scene, the resulting pixel is fully red. Due to the huge extent, 3D scenes can have the WPP channel should always be rendered in 32-bit floating-point to provide the accuracy needed.



XYZ Position

XY Disparity

XY Disparity is the only channel listed here that is not generated in a 3D application. These channels indicate where each pixel's corresponding matte can be found in a stereo image. Each eye, left and right, will use this vector to point to where that pixel would be in the other eye. This can be used for adjusting stereo effects, or to mask pixels in stereo space.



Combined X and Y Disparity channels

Using Cryptomatte in Fusion

Cryptomatte is an open-source technology that has been widely adopted in 3D applications. Unlike Z-Depth mattes or Object IDs, Cryptomatte automatically generates anti-aliased ID mattes from 3D renders with support for motion blur, transparency, and depth of field.

Fusion does not natively support the Cryptomatte format. However, using a free plugin from third-party developers, you can use Crypotmatte render passes in Fusion.

Crypotmatte for Fusion can be downloaded and installed for free:
https://github.com/Psyop/Cryptomatte

Or to use an easier installer, you can download Reactor, which comes bundled with Cryptomatte and offers many other free, useful Fusion plugins. Reactor can be found at:
https://www.steakunderwater.com

Propagating Auxiliary Channels

Ordinarily, auxiliary channels are propagated along with RGBA image data, from node to node, among gray-colored nodes, including those in the Blur, Filter, Effect, Transform, and Warp categories. Basically, most nodes that simply manipulate channel data propagate (and potentially manipulate) auxiliary channels with no problems.

However, when you composite two image layers using the Merge node, auxiliary channels only propagate through the image that's connected to the background input. The rationale for this is that in most composites that include computer-generated imagery, the background is most often the CG layer that contains auxiliary channels, while the foreground is a live-action green screen plate with subjects or elements that are combined against the background, which lack auxiliary channels.

Nodes That Use Auxiliary Channels

The availability of auxiliary channels opens up a world of advanced compositing functionality. This section describes every Fusion node that has been designed to work with images that contain auxiliary channels.

- **Copy Aux:** The Copy Aux tool can copy auxiliary channels to RGB and then copy them back. It includes some useful options for remapping values and color depths, as well as removing auxiliary channels.
- **Channel Booleans:** The Channel Boolean tool can be used to combine or copy the values from one channel to another in a variety of ways.
- **Custom Tool, Custom Vertex 3D, pCustom:** The "Custom" tools can sample data from the auxiliary channels per pixel, vertex, or particle and use that for whatever processing you would like.
- **Depth Blur:** The Depth Blur tool is used to blur an image based on the information present in the Z-Depth. A focal point is selected from the Z-Depth values of the image and the extent of the focused region is selected using the Depth of Field control. The Scale value default is based on an 8-bit image so it is important to lower the scale value when using the Depth Blur with 16- or 32-bit float files.
- **Disparity to Z, Z to Disparity, Z to WorldPos:** These tools use the inherent relationships between depth, position, and disparity to convert from one channel to another.
- **Fog:** The Fog tool makes use of the Z-Depth to create a fog effect that is thin closer to the camera and thickens in regions farther away from the camera. You use the Pick tool to select the Depth values from the image and to define the Near and Far planes of the fog's effect.

- **Lumakeyer:** The Lumakeyer tool can be used to perform a key on the Z-Depth channel by selecting the Z-Depth in the channel drop-down list.
- **Merge:** In addition to regular compositing operations, Merge is capable of merging two or more images together using the Z-Depth, Z-Coverage, and BG RGBA buffer data. This is accomplished by enabling the Perform Depth Merge checkbox from the Channels tab.
- **New Eye:** For stereoscopic footage, New Eye uses the Disparity channels to create new viewpoints or to transfer RGBA data from one eye to the other.
- **Shader:** The Shader tool applies data from the RGBA, UV and the Normal channels to modify the lighting applied to objects in the image. Control is provided over specular highlights, ambient and diffuse lighting, and position of the light source. A second image can be applied as a reflection or refraction map.
- **Shadow:** The Shadow tool can use the Z-Depth channel for a Z-Map. This allows the shadow to fall onto the shape of the objects in the image.
- **Smooth Motion:** Smooth Motion uses Vector and Back Vector channels to blend other channels temporally. This can remove high frequency jitter from problematic channels such as Disparity.
- **SSAO:** SSAO is short for Screen Space Ambient Occlusion. Ambient Occlusion is the lighting caused when a scene is surrounded by a uniform diffuse spherical light source. In the real world, light lands on surfaces from all directions, not from just a few directional lights. Ambient Occlusion captures this low frequency lighting, but it does not capture sharp shadows or specular lighting. For this reason, Ambient Occlusion is usually combined with Specular lighting to create a full lighting solution. The SSAO tool uses the Z-Depth channel but requires a Camera3D input.
- **Stereo Align:** For stereoscopic footage, the Disparity channels can be used by Stereo Align to warp one or both of the eyes to correct misalignment or to change the convergence plane.
- **Texture:** The Texture tool uses the UV channels to apply an image from the second input as a texture. This can replace textures on a specific object when used in conjunction with the Object ID or Material ID masks.
- **Time Speed and Time Stretcher:** These tools can use the Vector and BackVector channels to retime footage.
- **Vector Distortion:** The forward XY Vector channels can be used to warp an image with this tool.
- **Vector Motion Blur:** Using the forward XY Vector channels, the Vector Motion Blur tool can apply blur in the direction of the velocity, creating a motion blur effect.
- **Volume Fog:** Volume Fog is a raymarcher that uses the World Position channels to determine ray termination and volume dataset placement. It can also use cameras and lights from a 3D scene to set the correct ray start point and Illumination parameters.
- **Volume Mask:** Volume Mask uses the World Position channels to set a mask in 3D space as opposed to screen space. This allows a mask to maintain perfect tracking through a camera move.

TIP: The Object ID and Material ID auxiliary channels can be used by some tools in Fusion to generate a mask. The “Use Object” and “Use Material” settings used to accomplish this are found in the Settings tab of that node’s controls in the Inspector.

Image Formats That Support Aux Channels

Fusion supports auxiliary channel information contained in a variety of image formats. The number of channels and methods used are different for each format.

- **OpenEXR (*.exr):** The OpenEXR file format is the primary format used to contain an arbitrary number of additional image channels. Many renderers that will write to the OpenEXR format will allow the creation of channels that contain entirely arbitrary data. For example, a channel with specular highlights might exist in an OpenEXR. In most cases, the channel will have a custom name that can be used to map the extra channel to one of the channels recognized by Fusion.
- **SoftImage PIC (*.PIC, *.ZPIC and *.Z):** The PIC image format (used by SoftImage) is an older image format that can contain Z-Depth data in a separate file marked by the ZPIC file extension. These files must be located in the same directory as the RGBA PIC files and must use the same names. Fusion will automatically detect the presence of the additional information and load the ZPIC images along with the PIC images.
- **Wavefront RLA (*.RLA), 3ds Max RLA (*.RLA) and RPF (*.RPF):** This is an older image format capable of containing any of the image channels mentioned above. All channels are contained within one file, including RGBA, as well as the auxiliary channels. These files are identified by the RLA or RPF file extension. Not all RLA or RPF files contain auxiliary channel information, but most do. RPF files have the additional capability of storing multiple samples per pixel, allowing different layers of the image to be loaded for very complex depth composites.
- **Fusion RAW (*.RAW):** Fusion's native RAW format is able to contain all of the auxiliary channels as well as other metadata used within Fusion.

Creating Auxiliary Channels in Fusion

The following nodes create auxiliary channels:

- **Renderer 3D:** Creates these channels in the same way as any other 3D application would, and you have the option of outputting every one of the auxiliary data channels that the Fusion page supports.
- **Optical Flow:** Generates Vector and Back Vector channels by analyzing pixels over consecutive frames to determine likely movements of features in the image.
- **Disparity:** Generates Disparity channels by comparing stereoscopic image pairs.